

QUESTION 15

```
vedsk0831@DESKTOP-83FU7HB:~$ cd ~/oss
nano q15.c
vedsk0831@DESKTOP-83FU7HB:~/oss$ gcc -Wall -Wextra q15.c -o q15
vedsk0831@DESKTOP-83FU7HB:~/oss$ ./q15
Q15 completed successfully.
stdout and stderr were merged into combined.txt
vedsk0831@DESKTOP-83FU7HB:~/oss$ cat combined.txt
This is standard output.
This is standard error.
Command output
Command error
vedsk0831@DESKTOP-83FU7HB:~/oss$ ls
append.txt      output.txt      program6  q10      q13      q15
combined.txt    program3        program7  q10.c    q13.c    q15.c
errors.txt      program4        program8  q12      q14
input.txt       program5        program9  q12.c    q14.c
vedsk0831@DESKTOP-83FU7HB:~/oss$ -l
-l: command not found
vedsk0831@DESKTOP-83FU7HB:~/oss$ ls -l combined.txt
-rw-r--r-- 1 vedsk0831 vedsk0831 78 Sep 26 03:55 combined.txt
vedsk0831@DESKTOP-83FU7HB:~/oss$ |
```

CODE

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    int fd;
```

```
    pid_t pid;
```

```
    /*
```

```
    * Open combined.txt for writing.
```

```
    * O_TRUNC clears old contents.
```

```

*/
fd = open(
    "combined.txt",
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);

if (fd == -1)
{
    perror("Error opening combined.txt");
    return 1;
}

/* Create child process */
pid = fork();

if (pid < 0)
{
    perror("fork");
    close(fd);
    return 1;
}

/* Child process */
if (pid == 0)
{
    /*
     * Redirect stdout to combined.txt

```

```

*/
if (dup2(fd, STDOUT_FILENO) == -1)
{
    perror("dup2 stdout");
    exit(1);
}

/*
 * Redirect stderr to the SAME file
 */
if (dup2(fd, STDERR_FILENO) == -1)
{
    perror("dup2 stderr");
    exit(1);
}

/* Original descriptor no longer needed */
close(fd);

/*
 * Write to stdout
 */
dprintf(
    STDOUT_FILENO,
    "This is standard output.\n"
);

/*

```

```

    * Write to stderr

    */
    dprintf(
        STDERR_FILENO,
        "This is standard error.\n"
    );

    /*
    * Execute a command that produces
    * both normal output and an error.
    */
    execlp(
        "sh",
        "sh",
        "-c",
        "echo Command output; echo Command error >&2",
        NULL
    );

    /* Runs only if execlp fails */
    perror("execlp");
    exit(1);
}

/* Parent process */
close(fd);

/* Wait for child */

```

```
waitpid(pid, NULL, 0);
```

```
printf("Q15 completed successfully.\n");
```

```
printf("stdout and stderr were merged into combined.txt\n");
```

```
return 0;
```

```
}
```